

Windowed Minority Guidance: Preliminary Evidence for Timestep-Localized Effects in Diffusion Denoising

Vittoria Lanzo
Independent Researcher

Abstract

Minority guidance [4] steers diffusion sampling toward under-represented data regions via a classifier gradient at every denoising step. We test whether this effect is timestep-localized, comparing three equal-thirds windows—early $t \in [0, 333)$ (fine-detail, low noise), mid $t \in [333, 667)$, late $t \in [667, 1000)$ (coarse structure, high noise)—against full-chain and no-guidance baselines over 50 seeds on LSUN Bedroom (250 runs). Results confirm effects for mid, early, and full conditions vs. baseline (all $p < 0.001$). With late vs. baseline narrowly missing correction ($p = 0.00716$ vs. $\alpha/7 = 0.00714$). All windows produce detectable effects—preliminary evidence for mid-chain dominance without strict localization.

1 Introduction

Classifier-guided diffusion models [1] steer generation with classifier gradients at every denoising step. Um et al. [4] extend this to minority guidance, directing sampling toward underrepresented data regions. Because decreasing timesteps produce qualitatively different effects (Meng et al. [3]), the mid-chain phase may be most receptive to minority gradient signals—enabling cheaper inference-time guidance if guidance there recovers most of the full-chain effect. We test this directly: Section 4 reports findings that partially challenge it.

2 Experiment

We use the LSUN Bedroom 256×256 model with the frozen minority classifier of Um et al. [4] (ImageNet feature extractor = `mc_lsun.pt`, `guidance_scale=1.0`, `timestep_respacing=250`). Five conditions: baseline, minority-full $t \in [0, 1000)$, minority-early $t \in [0, 333)$, minority-mid $t \in [333, 667)$, minority-late $t \in [667, 1000)$. Guidance is rescaled to the active window only; results may not generalize. We run 50 seeds $\times$ 5 conditions = 250 runs (Kaggle P100, float32). Seeds 1–10 observed unbiased; results pre-registered in [4].

Primary metric: classifier cross-entropy loss at class 99 (lower = stronger guidance). Softmax confidence is corroborated only—distributions are severely right-skewed (mean/median: full 108x; late 154x; mid 46x; early 58x). Switching from pre-registration classifier confidence to loss is an unplanned procedural deviation; classifier-based effect sizes are one order of magnitude smaller.

Timestep convention. Window names follow the numerical magnitude of the timestep index t , not the chronological order of the DDPM reverse process [2]. “Early” ($t \in [0, 333)$) therefore corresponds to low-noise timesteps (fine-detail refinement); “late” ($t \in [667, 1000)$) to high-noise timesteps (coarse-structure formation).

Windowed `cond_fn`. Guidance is applied exclusively within the designated timestep window. Outside the window, the `cond_fn` is a no-op, returning the unmodified gradient zeroed out. Full details are given in Appendix A.2.

Table 1: Condition statistics ($n = 50$ seeds; scale = 1.0; LSUN Bedroom). Lower mean loss indicates stronger minority-class guidance. Win Rate is the fraction of seeds on which the condition beats baseline. Relative Effect = $(\text{baseline} - \text{cond}) / (\text{baseline} - \text{full})$; not a causal decomposition (windows interact across the rejection trajectory).

Condition	Mean Loss	CV	Win Rate	Relative Effect
baseline	11.978	0.19	—	0.000
minority-early	11.129	0.23	50/50	0.162
minority-mid	9.591	0.35	39/50	0.456
minority-late	10.717	0.30	34/50	0.241
minority-full	6.741	0.61	44/50	1.000

3 Results

Table 1 ($n = 50$; lower loss = stronger guidance).

The rank ordering mid > late > early > baseline holds across 50 seeds ($r_{\text{rb}} = 1.000, 0.812, 0.432, 0.925$ for early, mid, late, full vs. baseline respectively). Table 2 reports Wilcoxon signed-rank tests, Bonferroni threshold $\alpha/7 = 0.0071$.

Table 2: Wilcoxon signed-rank tests ($n = 50$ pairs; zero_method=zsplit). $r_{\text{rb}} \in [0, 1]$: rank-biserial correlation (effect size); $r_{\text{rb}} > 0$ indicates first condition has lower loss. Survives Bonferroni correction if $p < \alpha/7 = 0.0071$.

Comparison	W	p -value	r_{rb}
mid vs. baseline	120	< 0.0001	0.812
late vs. baseline	362	0.0072	0.432
early vs. baseline	0	< 0.0001	1.000
full vs. baseline	48	< 0.0001	0.925
mid vs. late	391	0.0166	0.387
late vs. early	547	0.3882	0.142
full vs. mid	181	< 0.0001	0.716

Mid recovers 0.456 of full-chain loss reduction (mean 9.591 vs. baseline 11.978 and full 6.741). Pre-registered criterion: Bonferroni $p < 0.0071$. Bootstrap 95% CI on mid mean: [8.67, 10.50]. The relative effect ($\Delta_{\text{rel}} = 0.456$) measures fraction of full-chain loss reduction recovered; the CI bounds are absolute loss units and are not directly comparable to Δ_{rel} .

Key finding: early-window guidance (operating at low noise, fine-detail phase) increases classifier confidence at every seed, demonstrating that guidance applied to near-clean images reliably improves minority-class quality ($r_{\text{rb}} = 1.000$, $W = 0$, $p < 0.0001$).

Late > Early at scale= 1.0: the mid-chain window separates mid-chain from coarse-structure guidance; the coarse-structure (late) window outperforms fine-detail (early) at this scale, though the difference is not statistically significant ($p = 0.388$).

4 Discussion and Conclusion

Mid-window guidance recovers 0.456 of the full-chain effect—the largest single-window contribution but insufficient alone. All three windows produce detectable seed-wise effects, contradicting binary localization. Key finding: early-window guidance increases classifier-measured guidance strength at every seed ($r_{\text{rb}} = 1.000$, 50/50 wins)—evidence that guidance applied to near-clean images reliably influences minority-class

sampling throughout the denoising trajectory. Mid is quantitatively dominant but the effect is distributed; mid alone recovers only 0.456 of the full-chain loss reduction and cannot substitute for full-chain guidance at this scale.

Limitations: single configuration (LSUN Bedroom, class 99, guidance_scale=1.0 vs. Um et al.’s scale 3.5; $n = 1$ sample per seed); high full-chain variance (CV= 0.61); no perceptual metrics (FID, precision/recall). Results may not generalize to other scales, datasets, or architectures; the multi-scale sweep in the extended report examines scale sensitivity directly.

Revised framing: mid is quantitatively dominant and the effect is distributed; no single window approximates full-chain performance. Late > early is consistent with high-noise guidance producing conflicting gradients before the denoising trajectory has committed to a minority-class layout.

References

- [1] Prafulla Dhariwal and Alex Nichol. Diffusion models beat GANs on image synthesis. In Advances in Neural Information Processing Systems (NeurIPS), volume 34, pages 8780–8794, 2021.
- [2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems (NeurIPS), volume 33, pages 6840–6851, 2020.
- [3] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. SDEdit: Guided image synthesis and editing with stochastic differential equations. In International Conference on Learning Representations, 2022.
- [4] Soobin Um, Suhyeon Lee, and Jong Chul Ye. Don’t play favorites: Minority guidance for diffusion models. In International Conference on Learning Representations (ICLR), 2024.

A Reproducibility

A.1 Experimental Configuration

Models and checkpoints (all publicly available):

Table 3: Model checkpoints used in all experiments.

Component	Checkpoint	Source
Diffusion model	lsun_bedroom.pt	OpenAI guided-diffusion
Feature extractor	256x256_classifier.pt	OpenAI guided-diffusion
Minority classifier	mc_lsun.pt	Um et al. (2024)

Shared hyperparameters (all fixed across conditions):

The minority classifier (mc_lsun.pt) expects 8×8 latent features produced by 256x256_classifier.pt with pool=True, in_channels=512, num_channels=100, classifier_width=128, classifier_pool=attention.

UNET.py patch—the upstream repository’s UNetModel.forward is patched. The following patch is required:

Listing 1: UNET.py patch: skip intermediate feature map during training.

```

1 def forward(self, x, timesteps, t_out=False):
2     # skipping feature map extracted until final return
3     if t_out:
4         return h
5     return self.out(h) # original logic

```

Data schema (data/experiment_runs.csv; 250 rows, one per run):

Table 4: Hyperparameters shared across all conditions and seeds.

Parameter	Value
diffusion_steps	1000
timestep_respadding	250
noise_schedule	linear
guidance_scale	1.0
image_size	256
num_channels	256
num_res_blocks	2
attention_resolutions	32,16,8
learn_sigma	True
use_fp16	False
manual_class_id	99
num_samples	1

run_id | condition | guidance_window | t_start | t_end | guidance_scale | seed | n_generated | timestep |
classifier_mean_loss | running_seconds

Metrics are computed at $t=0$ (final denoising step) by passing generated images through the frozen classifier chain. Each run produces one image; classifier_mean_ fields are single-sample values ($n=1$ per seed per condition).

A.2 Windowed Guidance Implementation

The windowed guidance requires no architectural modification—it is implemented entirely within the `cond_fn` callback:

Listing 2: Windowed `cond_fn`: applies classifier gradient only within $[t_{\text{start}}, t_{\text{end}}]$.

```

1  t_start = args.t_start    # e.g. 333 for mid window
2  t_end   = args.t_end     # e.g. 667 for mid window
3
4  def cond_func(x, t=None):
5      current_t = t[0].item()    # actual denoising timestep
6      # Zero grad outside the active window
7      if not (t_start <= current_t < t_end):
8          return torch.zeros_like(x)
9      with torch.enable_grad():
10         x_in = x.detach().requires_grad_(True)
11         logits = f_extractor(x_in, timesteps=t, t_out=True)
12         log_probs = F.log_softmax(logits, dim=-1)
13         selected = log_probs[range(log_probs.shape[0]), y.view(-1)]
14         # Scale applied only; sampler must NOT re-apply classifier_scale
15         return torch.autograd.grad(selected.sum(), x_in)[0] \
16             * args.classifier_scale

```

For baseline runs, `guidance_scale=0` is passed rather than gating by timestep. Window boundaries are raw diffusion timesteps (0–1000); the active window for mid is $t \in [333, 667]$, corresponding to 334 of the 1000 raw timesteps (≈ 134 of the 250 respaced steps).

A.3 Statistical Analysis Code

Listing 3: Statistical analysis: Wilcoxon signed-rank tests and rank-biserial correlation (n=50).

```

1 import csv, random
2 from scipy import stats
3
4 # --- load paired losses ---
5 data = {}
6 with open('data/experiment_runs.csv') as f:
7     for row in csv.DictReader(f):
8         data.setdefault(row['condition'], {})[(int(row['seed']))] = \
9             float(row['classifier_mean_loss'])
10
11 seeds = sorted(data['baseline'])
12 get = lambda cond: [data[cond][s] for s in seeds]
13 baseline, early, mid, late, full = (get(c) for c in
14     ['baseline', 'minority-early', 'minority-mid', 'minority-late', 'minority-full'])
15
16 # --- Wilcoxon signed-rank tests ---
17 def wilcoxon_rbc(a, b):
18     diffs = [ai - bi for ai, bi in zip(a, b)]
19     # zero_method='zsplit': split zero ranks equally between + and -
20     W, p = stats.wilcoxon(diffs, zero_method='zsplit')
21     # rank-biserial: W / (n*(n+1)/2) * 2 - 1 (normalized to [0,1])
22     n = len(diffs)
23     r_rb = 1 - 2 * W / (n * (n + 1) / 2)
24     return W, p, r_rb
25
26 comparisons = [
27     (mid, baseline, 'mid vs. baseline'),
28     (late, baseline, 'late vs. baseline'),
29     (early, baseline, 'early vs. baseline'),
30     (full, baseline, 'full vs. baseline'),
31     (mid, late, 'mid vs. late'),
32     (late, early, 'late vs. early'),
33     (full, mid, 'full vs. mid'),
34 ]
35
36 for a, b, label in comparisons:
37     W, p, r_rb = wilcoxon_rbc(a, b)
38     print(f'{label:<25} W={W:>5.0f} p={p:.4f} r_rb={r_rb:.3f}')
39
40 alpha_corrected = 0.05 / 7 # = 0.00714
41
42 # --- Bootstrap 95% CI on mid mean (10,000 resamples) ---
43 random.seed(42)
44 boot = sorted([sum(random.choices(mid, k=50)) / 50 for _ in range(10_000)])
45 ci = [boot[249], boot[9749]] # 2.5th and 97.5th percentiles
46
47 baseline_mean = sum(baseline) / len(baseline) # 11.978
48 threshold = 0.05 / 7 # 0.00714
49 mid_mean = sum(mid) / len(mid) # 9.591
50 # (misses by 0.20)

```

Environment: Python 3.11, scipy 1.x, numpy 2.x. All 250 runs generated on Kaggle (NVIDIA Tesla P100, float32, CUDA 11.x). The Kaggle notebook (including model download, patching, sampling loop, and CSV export) is available on request.